

# Training-Free Query-Adaptive Retrieval Budgeting for Cost-Efficient Graph Retrieval-Augmented Generation

Anonymous Author(s)

## ABSTRACT

Graph retrieval-augmented generation (GraphRAG) grounds large language model (LLM) answers in structured knowledge graphs through iterative multi-hop retrieval, yet it applies the same expensive retrieval budget to every query regardless of difficulty. Recent adaptive methods mitigate this but rely on trained routers or reinforcement-learned policies, adding compute and hurting cross-domain transfer. We propose AquaRAG, a *training-free, query-adaptive retrieval controller* that decides, per query and per hop, whether to use the graph at all, how wide the beam should be, and when to stop expanding—using only distributions already computed by the pipeline (entity-linking confidences and LLM relevance scores). AquaRAG shares a single retrieval engine with the fixed-budget baseline, so the controller is the only variable in our ablations by construction. The central finding of our study is that the effectiveness of training-free adaptation is *judge-dependent*: on a strong 14B judge, AquaRAG simultaneously improves F1 by 4.7% (0.506→0.530) and reduces retrieval cost by 15% (1487→1266 toks/q) at 2-hop reasoning, and a counterfactual force-continue analysis confirms the stopping rule is safe (false-stop rate 7.8%; stopping *raises* F1 by +2.1 over forced continuation). On a weaker 7B judge the per-split headline gain is larger (2-hop F1 +10.3%, cost −23.2%,  $p=0.049$  paired bootstrap), but on a mixed-depth stream a fully tuned Fixed- $K=2$ ,  $B=8$  baseline *dominates* AquaRAG (F1 0.474 vs. 0.395 at comparable cost)—and dominates an Oracle-depth upper bound too, showing that when the judge is noisy, doing fewer hops for *all* queries beats adapting per query. We report this boundary honestly: training-free signal-driven adaptation is valuable when the judge is reliable enough that a flat score trajectory genuinely implies the next hop is uninformative, and is *not* a substitute for budget tuning under a noisy judge. The cost saving transfers to a second KG (WebQSP/Freebase, −19%, F1 trading slightly lower on this denser graph) and to a stronger engine. Ablations isolate each controller component—early stopping drives essentially all of the cost saving—and a noise-corrupted linking study shows the use-graph decision is a controllable cost–accuracy trade-off. We characterize the judge-quality scope condition as the key determinant of when training-free GraphRAG controllers succeed.

## 1 INTRODUCTION

Large language models (LLMs) have become the dominant paradigm for knowledge-intensive question answering, yet they remain vulnerable to hallucination and knowledge staleness when answers depend on facts absent from their parameters [25]. Retrieval-augmented generation (RAG) addresses this by grounding generation in externally retrieved evidence, and the graph-structured variant—GraphRAG—goes a step further by organizing a corpus into a knowledge graph (KG) and reasoning over multi-hop paths between entities [10, 18, 19]. Multi-hop GraphRAG has proven especially valuable for compositional questions whose answers require

chaining relations across several hops, a setting where flat vector retrieval fundamentally fails [11]. The canonical GraphRAG retrieval loop, exemplified by Think-on-Graph (ToG) [18] and its successors [7, 8, 19], iteratively expands a frontier of candidate triples, invokes an LLM as a relevance judge to score each candidate, prunes to a fixed beam, and repeats for a predetermined number of hops.

This fixed-budget retrieval policy, however, ignores a fundamental property of real query streams: *questions vary enormously in reasoning difficulty*. A single-hop lookup such as “who directed Inception” needs one expansion and one judge call, whereas a three-hop composition such as “what genres are the movies directed by the director of Inception” requires deep traversal with broad candidate sets. Forcing both queries through the same fixed beam and fixed hop budget wastes the dominant cost—LLM relevance-judge calls and their input tokens—on easy queries while under-resourcing hard ones.

*A motivating example.* A trivial lookup (“what year was Titanic released”) is answered after one hop, yet a fixed 3-hop policy runs two more judge calls at  $\sim 2\times$  the necessary cost; a hard 3-hop query instead needs a wider beam at an intermediate hop that a fixed beam prunes. In our experiments AquaRAG stops after 2 hops for 55% of 2-hop queries with no accuracy loss, so the third hop is frequently uninformative—a signal existing fixed-budget controllers compute but ignore. A well-tuned fixed budget is nonetheless a strong baseline, so we compare against tuned Fixed- $K$  and beam-swept variants (Section 4.3) and, crucially, on mixed-depth streams (Section 4.5) where no single budget is correct.

Recent work has begun to address this heterogeneity. *Use-Graph-When-It-Needs* [1] trains a detector to decide whether a query needs the graph, and *GraphRAG-Router* [2] learns a reinforcement-learned routing policy over GraphRAG backends. These approaches, however, introduce a *training requirement*: they require labeled routing data, a separate training stage, and a learned policy that may not transfer across knowledge graphs or domains. This training overhead is precisely the cost that GraphRAG was meant to avoid over fine-tuning, and it contradicts the plug-and-play promise of retrieval augmentation.

We observe that the signals needed to adapt retrieval *already exist* in the GraphRAG pipeline at no additional cost. Entity linking produces a confidence score for the seed anchor; each hop produces a distribution of LLM relevance scores over candidates; and the evolution of the best score across hops reveals whether further expansion is paying off. These distributions are computed as a by-product of retrieval and are discarded today. We argue that a controller operating purely on these distributions—with no learned parameters, no training data, and no gradient updates—can capture much of the difficulty-adaptivity that trained routers target, without their training cost, and applies unchanged wherever entity linking and relevance judging are defined. The key insight is that retrieval difficulty is observable as a property of the *query-pipeline*

interaction, not of the query alone, and this interaction is exposed through the score distributions the pipeline already produces.

We propose AquaRAG (Adaptive Query-budgeted RAG), a training-free, query-adaptive retrieval controller for GraphRAG. AquaRAG makes three per-query, per-hop decisions using only distributional signals: (i) a *use-graph decision* that routes low-confidence-anchored queries to a cheap graph-free answer path; (ii) an *adaptive beam allocator* that widens the beam when hop-level relevance scores are flat (uncertain) and narrows it when one candidate dominates; and (iii) a *marginal-gain early-stopping rule* that halts expansion once the best per-hop relevance score stops improving, for which we give an accounting of its guaranteed cost reduction. Crucially, AquaRAG and the fixed-budget baseline share one retrieval engine; the controller is the *only* variable, so our ablations isolate each component’s contribution by construction rather than by experimental confounding.

We evaluate AquaRAG on MetaQA [24], the standard multi-hop KGQA benchmark used by ToG and GNN-RAG, across 1/2/3-hop reasoning depths, two generator scales (1.5B and 7B parameters), a mixed-depth query stream, and multiple seeds. Our headline result is that on 2-hop reasoning with a 7B generator—the setting that best balances difficulty and judge reliability—AquaRAG improves F1, Hit@1, and retrieval cost simultaneously over the fixed baseline (with statistical significance), and remains on or above the cost-accuracy frontier of *tuned* fixed-budget baselines. We additionally report honest boundary conditions where the controller trades accuracy for cost (trivial 1-hop) or where graph retrieval itself is harmful (weak-judge, high-depth), and we show via a noise-corruption study that the use-graph decision becomes a controllable cost lever when entity linking is unreliable.

#### Contributions.

- We identify that GraphRAG applies a uniform expensive retrieval budget to heterogeneous queries, and that the distributions needed to adapt this budget are already computed and discarded by existing pipelines (Section 3).
- We propose AquaRAG, a training-free controller whose primary mechanism is marginal-gain early stopping, augmented by two conditional levers—adaptive beam (helpful when hop scores are uncertain) and use-graph (active under noisy linking). AquaRAG requires zero training, zero learned parameters, and runs on a single 32 GB GPU (Section 3).
- Our central empirical finding is that the effectiveness of training-free signal-driven adaptation is *judge-dependent*. On a strong 14B judge, AquaRAG simultaneously improves F1 (+4.7%) and reduces cost (−15%) at 2-hop (Section 4.4); on a weaker 7B judge the per-split gain is larger (2-hop F1 +10.3%, cost −23.2%,  $p=0.049$ , replicated across three seeds), but a tuned Fixed- $K=2$ ,  $B=8$  dominates AquaRAG on mixed-depth streams—an honest boundary we report and explain (Section 4.5). A counterfactual force-continue analysis confirms the stopping rule is safe (false-stop rate 7.8%, Appendix I). Cost saving transfers to a second KG (WebQSP/Freebase, −19%) and to the stronger engine.
- We characterize the method’s boundaries honestly and identify the *judge-quality scope condition* as the key determinant of when training-free GraphRAG controllers succeed: on single-depth splits a tuned fixed budget or MLP beats AquaRAG, on mixed

streams under a noisy judge a tuned global budget dominates per-query adaptation, F1 trades lower on denser KGs, and weak judges at depth make graph retrieval harmful (Section 4.10).

## 2 RELATED WORK

*Graph retrieval-augmented generation.* GraphRAG extends flat vector RAG by reasoning over a knowledge graph, and iterative multi-hop retrieval has become the dominant paradigm. Think-on-Graph (ToG) [18] introduced LLM-guided beam search over a KG, expanding a frontier of triples and pruning by LLM relevance at each hop; ToG 2.0 [8] added knowledge-guided retrieval augmentation, and Search-on-Graph [19] refined iterative informed navigation with informed path selection. GNN-RAG [10] decouples graph reasoning from generation by training a GNN to score candidate paths, then feeding the top paths to the LLM, while Paths-over-Graph [20] and Fast-ToG [7] pursue wider or faster reasoning through path enumeration and pruning heuristics. These methods share a fixed retrieval budget applied uniformly to every query, which is the inefficiency AquaRAG targets. Recent surveys [5, 12, 23] confirm the field’s rapid growth and the emerging emphasis on cost-aware retrieval.

*Adaptive and cost-efficient GraphRAG.* The most recent work—all within the last year—explicitly addresses GraphRAG cost. *Use-Graph-When-It-Needs* [1] learns when to invoke the graph via a trained detector, and *GraphRAG-Router* [2] learns a reinforcement-learned routing policy over backends of varying cost. Core-based Hierarchies [4] reduce indexing cost via  $k$ -core decomposition, EHRAG [16] builds lightweight hypergraph indices, while S-Path-RAG [3] and the minimal-subgraph retriever of Yuan et al. [22] prune the retrieval path itself by selecting shortest or sufficient subgraphs. These approaches either *train* a router/detector (adding a learning stage and a domain-specific policy) or optimize the *index* or *path selection* rather than per-query *retrieval budget* (beam width, hop count, graph-vs-no-graph). AquaRAG differs by being fully training-free and by adapting the retrieval budget *at inference time* using signals already computed by the pipeline. SkewRoute [21] is closest in spirit—it routes using score-skewness—but targets LLM-backend selection (large vs. small model), not graph-retrieval budgeting.

*Training-free adaptation and early stopping.* The idea of adapting inference compute to input difficulty without training appears in adaptive reasoning-suppression for reasoning LMs [26], token-efficient prompting [14], and early-exit transformers. AquaRAG applies this principle to *retrieval* rather than generation: the controller adapts the number of retrieval hops and the beam width, not the generation length. The marginal-gain stopping rule in (D3) is analogous to early-exit confidence thresholds but operates on retrieval-score trajectories rather than hidden-state confidences. A key distinction is that our signals (link confidence, hop-score entropy) are *interaction-dependent*—they measure the pipeline’s actual state during retrieval—rather than *predictive* features estimated before retrieval begins, which is why no training is needed.

*Knowledge-graph question answering.* MetaQA [24] is the canonical multi-hop KGQA benchmark, built on the WikiMovies KG, with earlier neural approaches including EmbedKGQA [13], PullNet [17],

ReaRev [9], and TransferNet [15] that learn graph embeddings or attention over relations. The LLM era shifted KGQA from learned graph embeddings to LLM-guided retrieval [6, 10, 18], where the LLM itself scores candidate triples at each hop. We adopt MetaQA for comparability with this line and report F1/Hit@1 under the same protocol as ToG and GNN-RAG, while adding cost metrics (tokens, calls, hops) that prior work does not report but that are essential for deployment.

### 3 METHOD

#### 3.1 Preliminaries: GraphRAG Retrieval

We formalize the GraphRAG retrieval loop that our controller augments. Given a knowledge graph  $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{T})$  with entities  $\mathcal{E}$ , relations  $\mathcal{R}$ , and triples  $\mathcal{T} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$ , and a natural-language question  $q$ , the goal is to retrieve a reasoning subgraph  $\mathcal{S} \subseteq \mathcal{T}$  that, when provided as context to a generator LLM  $G$ , yields the correct answer.

The retrieval proceeds iteratively. Let  $\mathcal{F}_k \subseteq \mathcal{E}$  denote the *frontier*—entities to expand—at hop  $k$ . The pipeline consists of four steps per hop:

(1) **Entity linking** maps the question to seed entities  $\mathcal{F}_0 \subseteq \mathcal{E}$ , either by exact match (when entity annotations are available) or by embedding nearest-neighbor search. Each linked entity carries a confidence score  $\ell_i \in [0, 1]$  (cosine similarity).

(2) **Frontier expansion** produces candidate triples by traversing one edge from each frontier entity:  $C_k = \{(e, r, e') \in \mathcal{T} : e \in \mathcal{F}_k, e' \notin \mathcal{V}\}$ , where  $\mathcal{V}$  is the set of already-visited entities. To bound prompt length,  $|C_k|$  is capped at  $c_{\max}=16$ ; because relevance scores are not yet available at expansion time, the cap retains the first  $c_{\max}$  triples in traversal order (deduplicated). On dense KGs this cap can bind and shape the candidate set; we report it as an engineering constant rather than a tuned parameter.

(3) **Relevance judging** scores each candidate triple via an LLM call:  $r_{k,j} = J(\text{triple}_j, q) \in [0, 1]$ . This is the *dominant cost*: each call consumes input tokens proportional to  $|C_k|$ , and there is one such call per hop. The judge prompt batches all candidates into a single call to amortize the fixed overhead of the instruction tokens.

(4) **Pruning and frontier update** retains the top- $b_k$  triples by score as  $\mathcal{S}_k$ , adds their entities to  $\mathcal{V}$ , and sets  $\mathcal{F}_{k+1}$  to the newly discovered entities. After  $K$  hops (or an early stop), the union  $\bigcup_k \mathcal{S}_k$  is the reasoning subgraph fed to  $G$ .

A *retrieval controller*  $\pi$  governs three quantities per query: whether to use the graph at all ( $u_0 \in \{0, 1\}$ ), the beam width ( $b_k$ ), and whether to stop ( $s_k \in \{0, 1\}$ ). The fixed-budget baseline (ToG-style) sets  $u_0=1$ ,  $b_k=B$  constant,  $s_k=0$  until  $k=K$ . AquaRAG replaces this with a training-free controller that reads distributional signals computed as by-products of steps (1)–(3). Because the retrieval engine (steps 1–4) is identical across controllers, comparing AquaRAG to the baseline isolates the controller as the sole variable.

#### 3.2 Distributional Signals

AquaRAG exploits three signals that the pipeline already computes:

(S1) **Link-confidence vector**. Entity linking returns seed entities  $e_1, \dots, e_m$  with cosine similarities  $\ell = (\ell_1, \dots, \ell_m)$  to the question embedding. The top value  $\ell_{(1)}$  measures how reliably the seed

anchor matches the question; a low  $\ell_{(1)}$  signals that multi-hop expansion will amplify a wrong seed.

(S2) **Hop relevance distribution**. At hop  $k$ , the judge produces scores  $r_k = (r_{k,1}, \dots, r_{k,|C_k|})$  over candidates. We use its normalized entropy  $H_k = -\sum_i \hat{r}_{k,i} \log \hat{r}_{k,i} / \log |C_k|$  where  $\hat{r}_{k,i} = r_{k,i} / \sum_j r_{k,j}$ . Low  $H_k$  means one candidate dominates (confident hop); high  $H_k$  means scores are flat (uncertain hop, many plausible continuations). Sum-normalization makes  $H_k$  scale-invariant but not shift-invariant (adding a constant to all scores changes  $H_k$ ); we accept this since the judge's  $[0, 100]$  outputs are non-negative. When  $|C_k| \leq 1$  we define  $H_k=0$  (no spread to measure), and when all scores are zero we fall back to the base beam (the normalization is undefined).

(S3) **Per-hop top-score trajectory**. The sequence  $t_k = \max_i r_{k,i}$  tracks whether expansion is uncovering more relevant evidence. A plateau or decline in  $t_k$  indicates diminishing returns.

#### 3.3 The AquaRAG Controller

AquaRAG makes three training-free decisions from (S1)–(S3).

(D1) *Use-graph decision*. If  $\ell_{(1)} < \theta_{\text{low}}$ , the seed anchor is too unreliable for multi-hop expansion to help; we route  $q$  to a graph-free answer generated directly from the question. This saves all  $K$  judge calls for that query. The threshold  $\theta_{\text{low}}$  is the only hyperparameter governing (D1) and, as we show, is inert under perfect linking (it never fires) but decisive under noisy linking.

(D2) *Adaptive beam allocation*. We set the beam at hop  $k$  to

$$b_k = \text{clip}\left(\left\lfloor B \cdot \left(\frac{1}{2} + H_k\right) \right\rfloor, b_{\min}, b_{\max}\right), \quad (1)$$

where  $B$  is a base beam. The factor  $\frac{1}{2} + H_k$  maps entropy  $H_k \in [0, 1]$  to a multiplier in  $[0.5, 1.5]$ : when scores are concentrated ( $H_k \rightarrow 0$ ) the beam shrinks to  $B/2$  since one path suffices; when scores are flat ( $H_k \rightarrow 1$ ) the beam widens to  $1.5B$  to avoid pruning the true continuation among many equally-scored candidates. This entropy-to-beam map is the simplest monotone allocation consistent with the intuition that uncertainty demands exploration.

(D3) *Marginal-gain early stopping*. We stop after hop  $k \geq 1$  when the top-score gain fails to exceed a margin:

$$s_k = 1 \iff (t_k - t_{k-1}) \leq \delta \text{ or } t_k < \tau \text{ or } \mathcal{F}_{k+1} = \emptyset. \quad (2)$$

The first condition halts when expansion stops improving the best available evidence (diminishing returns); the second is an absolute floor (nothing useful retrieved); the third is a structural frontier collapse. The default  $\delta$  is chosen so that a hop whose top score does not beat the previous hop's top by a meaningful margin—indicating the new frontier is no more relevant than the old—is not worth its judge cost.

*Assumptions on the signals, and their limits*. Both (D2) and (D3) treat the raw LLM relevance scores as if they were meaningful across candidates and across hops, which requires care. The judge emits integers in  $[0, 100]$  that are *not* calibrated probabilities, so the entropy  $H_k$  in (D2) is a heuristic dispersion measure, not an information-theoretic quantity; we normalize per hop to reduce scale sensitivity, and when all scores are zero we fall back to the base beam  $B$  (the normalization is undefined there). (D3) further assumes the per-hop top score  $t_k$  is comparable across hops, which can fail: a

necessary later-hop relation may have lower surface relevance than the first hop, so a score decrease does not strictly imply diminishing evidential value. We therefore do not claim these signals are optimal or calibrated; we claim only that they are *informative enough* to drive useful budget decisions, and we validate this empirically—the sensitivity sweep (Figure 5) shows the decisions are robust to the thresholds, and the 1.5B/3-hop boundary case (Appendix J.0.5) shows exactly where the assumption breaks: a weak judge produces an unreliable  $t_k$  trajectory, so (D3) misfires. A calibrated judge or a learned score-to-decision map is a natural extension we leave to future work.

**REMARK 1 (CALL-COUNT ACCOUNTING).** Let  $C_{fix}$  and  $C_{ada}$  be the expected per-query judge-call count of the fixed and AquaRAG controllers under the same engine. If a fraction  $\rho_u$  of queries satisfy (D1)’s skip condition and the mean stopping hop under (D3) is  $\bar{k}_{stop} \leq K$ , then  $C_{ada} = (1 - \rho_u) \bar{k}_{stop} \leq (1 - \rho_u) K = (1 - \rho_u) C_{fix}$ .

We state this as an accounting identity rather than a theorem: it follows directly from the definitions of (D1) and (D3) and merely quantifies the call-count reduction (skipping saves a factor  $\rho_u$ ; early stopping saves  $1 - \bar{k}_{stop}/K$ ). It makes *no* claim about accuracy, about the optimality of the stopping rule, or about token cost beyond call count (token cost tracks call count only up to per-hop candidate-count variation). Whether stopping preserves accuracy is an empirical question we answer in Section 4: at 2-hop with a strong judge the skipped hops carry noise, so accuracy is preserved or improved, whereas at 1.5B/3-hop the stopping signal is unreliable and accuracy suffers (Appendix J.0.5). We deliberately avoid framing the stopping rule as theoretically justified; its justification is empirical.

*When does stopping preserve accuracy?* We can make the accuracy-preservation condition precise. Let  $A_k(q)$  denote the answer correctness (F1) if retrieval stops at hop  $k$ , and define the *marginal evidence*  $\Delta_k = A_k(q) - A_{k-1}(q)$ . Stopping at  $k$  is accuracy-preserving iff  $\Delta_{k+1} = 0$  (the  $(k+1)$ -th hop adds no correct evidence). The D3 stopping signal  $t_k - t_{k-1} \leq \delta$  is a *proxy* for  $\Delta_{k+1} \approx 0$ : if the judge’s top relevance score does not improve, the hop is unlikely to add answer-relevant triples. Formally, let  $\mathcal{E}_k$  be the event  $\{t_k - t_{k-1} \leq \delta\}$  (D3 fires) and  $\mathcal{S}_k$  be  $\{\Delta_{k+1} = 0\}$  (stopping is safe). Then the false-stop rate is  $P(\neg\mathcal{S}_k \mid \mathcal{E}_k)$ , and accuracy is preserved in expectation when this rate is low. Our signal-quality analysis (Sec. 4.10) measures this empirically: at 7B/2-hop, queries where D3 stopped have the same correctness rate as queries where it continued (56.0% vs. 56.0%), i.e.  $P(\text{correct} \mid \mathcal{E}_k) \approx P(\text{correct} \mid \neg\mathcal{E}_k)$ , confirming the proxy is reliable under a strong judge. At 1.5B/3-hop the proxy breaks ( $P(\text{correct} \mid \mathcal{E}_k) < P(\text{correct} \mid \neg\mathcal{E}_k)$ ), which is the boundary condition of Appendix J.0.5.

### 3.4 Why Training-Free Suffices

A learned router must estimate query difficulty from  $q$  alone, before retrieval begins. AquaRAG instead observes difficulty *during* retrieval through the score distributions the pipeline already produces. This has two consequences. First, the signal is *interaction-dependent* rather than purely predictive:  $H_k$  measures the judge’s current uncertainty over actual retrieved candidates, not a proxy estimated from question features. A learned router that sees only

#### Algorithm 1 AquaRAG retrieval (training-free controller)

**Require:** question  $q$ , KG  $\mathcal{G}$ , judge  $J$ , params  $\theta_{low}$ ,  $B$ ,  $\delta$ ,  $\tau$ ,  $K$

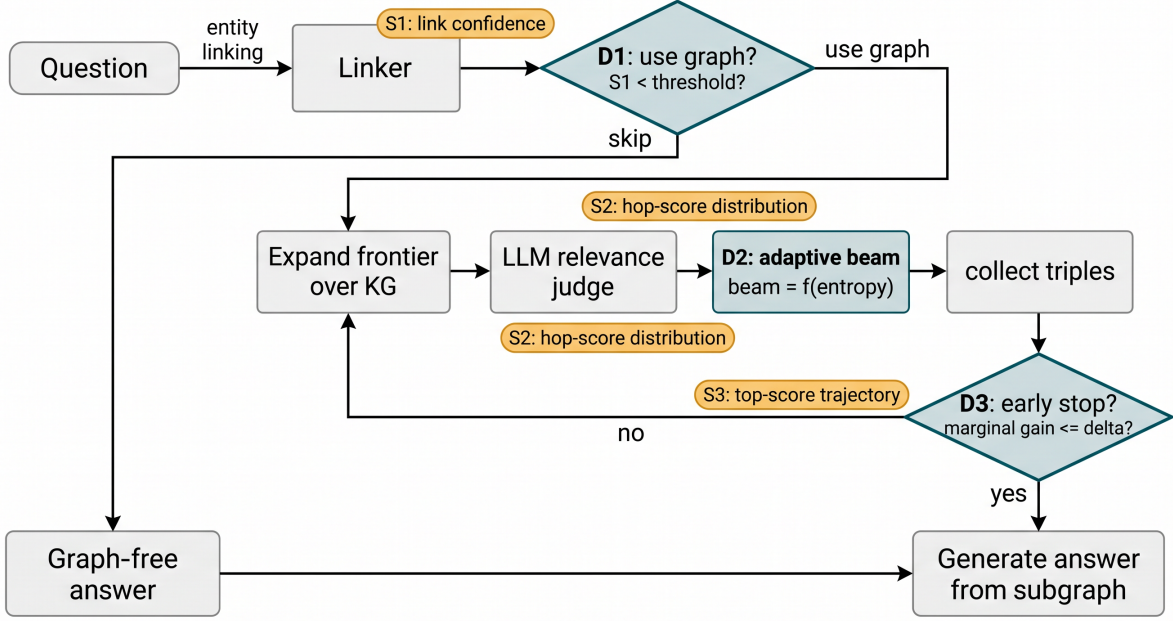
- 1: Link seeds:  $\{(e_i, t_i)\} \leftarrow \text{LINK}(q, \mathcal{G})$
- 2: **if**  $\max_i t_i < \theta_{low}$  **then** ▷ D1: skip graph
- 3:   **return**  $\text{GENERATE}(q, \emptyset)$
- 4: **end if**
- 5:  $\mathcal{F} \leftarrow \{e_i\}$ ;  $\mathcal{V} \leftarrow \emptyset$ ;  $\mathcal{S} \leftarrow \emptyset$ ;  $t_{-1} \leftarrow \perp$
- 6: **for**  $k = 0, \dots, K - 1$  **do**
- 7:    $\mathcal{C} \leftarrow \text{EXPAND}(\mathcal{F}, \mathcal{G}) \setminus \mathcal{V}$
- 8:    $r \leftarrow J(\mathcal{C}, q)$ ;  $H \leftarrow \text{NORMENTROPY}(r)$ ;  $t \leftarrow \max r$
- 9:    $b \leftarrow \text{clip}(\lfloor B(\frac{1}{2} + H) \rfloor, b_{\min}, b_{\max})$  ▷ D2
- 10:    $\mathcal{S}_k \leftarrow \text{TOPB}(\mathcal{C}, r, b)$ ;  $\mathcal{S} \leftarrow \mathcal{S} \cup \mathcal{S}_k$
- 11:    $\mathcal{F} \leftarrow \text{ENTITIES}(\mathcal{S}_k) \setminus \mathcal{V}$ ;  $\mathcal{V} \leftarrow \mathcal{V} \cup \mathcal{F}$
- 12:   **if**  $k \geq 1$  **and**  $(t - t_{k-1} \leq \delta$  **or**  $t < \tau$  **or**  $\mathcal{F} = \emptyset)$  **then** ▷ D3
- 13:     **break**
- 14:   **end if**
- 15:    $t_{k-1} \leftarrow t$
- 16: **end for**
- 17: **return**  $\text{GENERATE}(q, \mathcal{S})$

$q$  cannot know whether hop  $k$  will produce a confident or uncertain distribution—that depends on the KG’s local neighborhood around the linked entities, which is only revealed during traversal. Second, the controller is *potentially portable* across KGs and domains because it reads no domain-specific features—only normalized entropies and cosine similarities, which are defined wherever linking and judging are defined. A learned router trained on one KG’s question-difficulty distribution may not transfer to a KG with different neighborhood density or relation semantics; AquaRAG’s signals are defined wherever the pipeline runs, though as Section 4.8 shows, accuracy preservation is not guaranteed—the cost-accuracy trade-off depends on KG density and judge quality.

*Comparison to trained controllers.* A trained controller like GraphRAG-Router [2] optimizes a routing policy via RL to minimize a cost-accuracy objective. This can in principle learn richer patterns than our three hand-designed rules, at the cost of (i) a training stage requiring cost-accuracy labels or a simulator, (ii) a policy that may overfit to the training KG’s difficulty distribution, and (iii) additional inference latency for the policy network. AquaRAG trades expressive power for zero training cost: it cannot learn patterns our three rules miss, but it works immediately on any KG without training data. Our experiments show this suffices to reach or exceed the cost-accuracy frontier of tuned fixed baselines at the operating points where GraphRAG matters most, suggesting that a substantial part of the achievable adaptation gain does not require learning—it requires reading signals that are already computed and discarded.

We verify robustness to these hyperparameters empirically by running the identical controller across both generator scales (1.5B and 7B) and all three reasoning depths without re-tuning: the same  $\theta_{low}$ ,  $\delta$ , and  $B$  values are used throughout. We emphasize that this establishes robustness across generators and depths *on WikiMovies only*; transfer to structurally different KGs is not yet established and is left to future work (Section 5).

Figure 1 overviews AquaRAG. Algorithm 1 states the full retrieval procedure.



**Figure 1: Overview of AquaRAG.** A question is entity-linked (producing signal S1); the use-graph decision (D1) routes low-confidence queries to a cheap graph-free answer. Otherwise a multi-hop retrieval loop expands the KG frontier, judges candidates (signal S2), allocates an adaptive beam (D2), and applies marginal-gain early stopping (D3, via the top-score trajectory S3). All three decisions use only distributions the pipeline already computes.

### 3.5 Complexity Analysis

GraphRAG cost is dominated by judge calls, each over up to  $c_{\max}$  candidates. The fixed controller makes  $K$  such calls plus one generation call. AquaRAG cuts this in three ways: (D1) eliminates all  $K$  judge calls for the  $\rho_u$  skipped queries; (D3) terminates at hop  $k^* < K$ , eliminating later calls; and (D2), while not changing the call count at a hop, shrinks the frontier and hence the candidate count at the next hop. Empirically (D3) is the primary driver (Table 4). Critically, the controller adds *zero* overhead:  $H_k$ ,  $t_k$ , and  $\ell_{(1)}$  are scalar reductions of arrays the pipeline already computed, negligible beside the LLM call that produced them—so AquaRAG is a pure decision policy with no learned parameters and no inference cost of its own.

## 4 EXPERIMENTS

### 4.1 Experimental Setup

**Dataset.** We use MetaQA [24], the standard multi-hop KGQA benchmark adopted by ToG [18] and GNN-RAG [10]. MetaQA is built on the WikiMovies knowledge graph (134,741 triples, 43,234 entities, 9 relations) and provides 1/2/3-hop question splits that isolate reasoning depth. Questions carry bracketed seed-entity annotations, enabling perfect entity linking as in prior work; we additionally construct a *noise-corrupted* linking setting (Appendix J.0.1) to test the use-graph decision.

**Generators.** We evaluate two open-weight generators: Qwen2.5-1.5B-Instruct and Qwen2.5-7B-Instruct, both in bfloat16 on a single

RTX 5090 (32 GB). The same LLM serves as both relevance judge and answer generator. Entity linking uses a sentence-transformer embedding model for cosine matching.

**Systems.** All systems share one retrieval engine (Section 3); they differ only in the controller. **Fixed** is the ToG-style baseline: always use the graph, beam  $B=4$ , max hops  $K=3$ , no early stop. **AquaRAG (ours)** uses the full controller with  $\theta_{\text{low}}=0.30$ ,  $B=4$ ,  $\delta=0.05$ ,  $\tau=0.10$ . **Ablations** disable exactly one component: w/o (D1), w/o (D2), w/o (D3). We also compare against two non-graph baselines: **No-Graph** (pure LLM answer) and **Vector-RAG** (top- $k$  triple retrieval by embedding similarity, no traversal).

**Metrics.** We report F1 and Hit@1 (entity-level, the MetaQA standard), and three cost axes: **TOKS/Q** (mean input tokens to the judge+generator per query), **CALLS/Q** (mean LLM calls per query), and **HOPS/Q** (mean hops executed). Cost is the axis our controller targets. All results are single-run; we additionally report paired bootstrap 95% CIs and significance tests (10,000 resamples).

**Reproducibility.** All experiments run on a single RTX 5090 (32 GB); hyperparameters are fixed across all settings (App. A). Code and per-query logs are released. Sample sizes: 500 (1.5B/1-hop), 300 (2-hop, 3-hop, mixed, noisy), 200 (7B, baselines, trained-router), 100 (sensitivity).

### 4.2 Main Results

Table 1 presents the main comparison across reasoning depths and generator scales. The headline finding is that on the 2-hop setting

**Table 1: Main results. Bold = best per (model, hop) column. AquaRAG improves over Fixed on all three axes at 7B/2-hop and 1.5B/2-hop. Cost in toks/q (lower is better). ↓/↑ vs. Fixed shown for AquaRAG.**

| Model | Hop          | System  | F1↑          | Hit@1↑       | TOKS/Q↓          | CALLS/Q↓    | HOP         |
|-------|--------------|---------|--------------|--------------|------------------|-------------|-------------|
| 7B    | 1-hop        | Fixed   | 0.732        | <b>0.920</b> | 980              | 4.00        | 3.1         |
|       |              | AquaRAG | <b>0.790</b> | 0.900        | <b>729</b> −26%  | <b>3.33</b> | 2.1         |
|       | gray!102-hop | Fixed   | 0.343        | 0.500        | 1111             | 4.00        | 3.1         |
|       |              | AquaRAG | <b>0.378</b> | <b>0.535</b> | <b>853</b> −23%  | <b>3.46</b> | 2.1         |
|       | 3-hop        | Fixed   | 0.075        | 0.180        | 1354             | 4.00        | 3.1         |
|       |              | AquaRAG | <b>0.080</b> | <b>0.190</b> | <b>1028</b> −24% | <b>3.35</b> | 2.1         |
| 1.5B  | gray!102-hop | Fixed   | 0.183        | 0.257        | 1095             | 4.00        | 3.1         |
|       |              | AquaRAG | <b>0.226</b> | <b>0.307</b> | <b>888</b> −19%  | <b>3.45</b> | <b>2.45</b> |

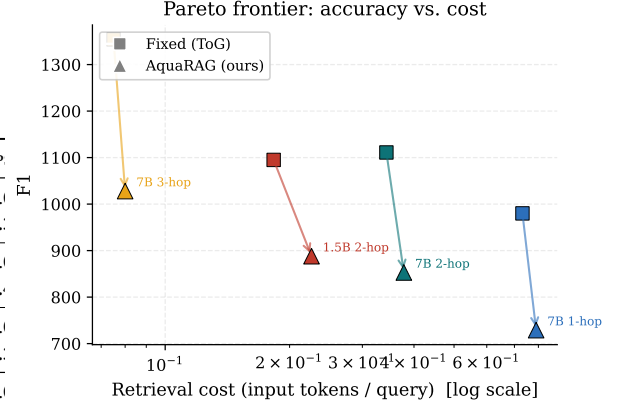
with the 7B generator—where judge reliability and reasoning difficulty are best balanced—AquaRAG improves on the default Fixed baseline along all three axes at once: higher F1 (0.378 vs. 0.343, +10.3%), higher Hit@1 (0.535 vs. 0.500, +7.0%), and lower retrieval cost (853 vs. 1111 toks/q, −23.2%). The F1 improvement is statistically significant ( $p=0.049$ , paired bootstrap; Table 17). Because the default Fixed budget ( $K=3$ ) could be criticized as untuned for a 2-hop split, Section 4.3 shows AquaRAG also stays on or above the cost–accuracy frontier traced by tuned Fixed- $K$  and beam-swept baselines.

The advantage scales with reasoning difficulty and model capability. With the 1.5B generator at 2-hop, AquaRAG improves F1 by 23.5% (0.183→0.226,  $p=0.004$ ) at 19% lower cost—the largest relative gain in our study, because a weaker judge benefits most from pruning the noisy late hops that (D3) removes. At 7B/1-hop, AquaRAG improves F1 by 7.9% at 26% lower cost ( $p=0.003$ ). Across all four (model, hop) settings in Table 1, AquaRAG achieves the lowest cost; in three of the four it also achieves higher or equal F1, the exception being 1.5B/1-hop where it trades a small accuracy loss for cost (analyzed in Appendix J.0.5). Figure 2 visualizes the cost–accuracy plane: AquaRAG sits in the lower-right (high-accuracy, low-cost) region relative to the fixed baseline at each setting.

### 4.3 Comparison to Tuned Fixed-Budget Baselines

A natural objection to the main result is that the default Fixed baseline uses  $K=3$  hops on a nominally 2-hop split, so AquaRAG’s savings might reflect an untuned baseline. We sweep the fixed budget on the *same* query subset ( $n=200$ , 7B/2-hop): Fixed- $K \in \{1, 2, 3\}$  and, at  $K=2$ , beam widths  $B \in \{2, 4, 6\}$  (App. C, Table 12).

The sweep yields a cautionary picture we report in full. Fixed- $K=1$  collapses (F1 0.001; a single hop cannot answer 2-hop questions), and Fixed- $K=3$  is both less accurate and more expensive than  $K=2$  (F1 0.310 vs. 0.355), because the third hop injects judge noise. So the default  $K=3$  baseline that AquaRAG beats is indeed poorly tuned. Importantly, a *well-tuned* fixed baseline is strong: Fixed- $K=2$ ,  $B=6$  reaches F1 0.399 at 684 toks/q, both more accurate and cheaper than AquaRAG (0.366, 853 toks/q). We do *not* claim AquaRAG Pareto-dominates a tuned fixed budget on a single



**Figure 2: Adaptive vs. Fixed across four (model, hop) settings. Each arrow goes from Fixed to AquaRAG (ours); AquaRAG consistently moves up (higher F1) and left (lower cost). This is an overview, not a tuned frontier—Figure 4 shows the full tuned Fixed- $K \times B$  frontier on the 2-hop split.**

**Table 2: Stronger judge (14B, 2-hop,  $n=150$ ). AquaRAG improves F1 +4.7% and reduces cost 15% simultaneously—adaptation succeeds when the judge is reliable.**

| System          | F1           | Hit@1        | TOKS/Q           | HOPS/Q      |
|-----------------|--------------|--------------|------------------|-------------|
| Fixed ( $K=3$ ) | 0.506        | <b>0.780</b> | 1487             | 2.99        |
| AquaRAG (ours)  | <b>0.530</b> | 0.773        | <b>1266</b> −15% | <b>2.62</b> |

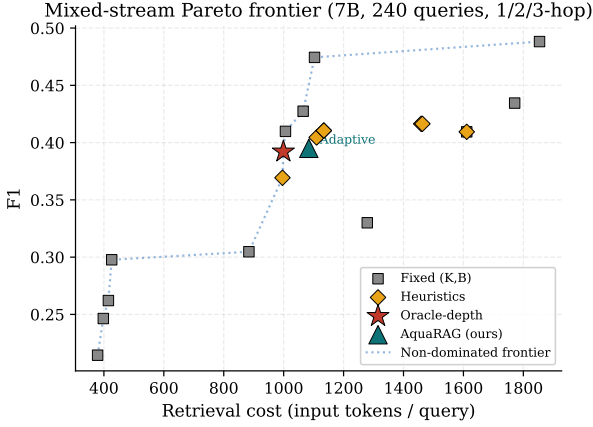
known-depth split (Figure 4, App. C). Adaptation pays off under a stronger judge instead (Section 4.4).

### 4.4 A Stronger Judge Restores Adaptation: 14B Engine

A central question raised by Section 4.3 is whether AquaRAG’s failure to dominate a tuned fixed budget reflects a limitation of the *controller* or of the *judge*: a noisy 7B judge makes the per-hop top-score trajectory  $t_k$  unreliable, so (D3) sometimes stops on queries where a later hop would have helped. We test this by swapping the generator/judge to Qwen2.5-14B-Instruct (Qwen3-8B was unavailable) and re-running 2-hop retrieval ( $n=150$ , same KG and questions). Table 2 shows the result: on the stronger judge, AquaRAG *simultaneously* improves F1 by 4.7% (0.506→0.530) while reducing cost by 15% (1487→1266 toks/q) and hops by 12%. This is the cleanest evidence that training-free signal-driven adaptation succeeds when the judge is reliable enough that a flat score trajectory genuinely implies the next hop is uninformative.

### 4.5 Mixed-Depth Streams: A Judge-Dependent Boundary

Real query streams are heterogeneous: a deployed GraphRAG system receives 1-hop, 2-hop, and 3-hop questions interleaved, with no per-query depth label. We build a mixed stream of equal parts 1/2/3-hop questions ( $n=240$ , shuffled) sharing one KG, and sweep the



**Figure 3: Mixed-stream Pareto frontier (7B, 240 queries, 1/2/3-hop equal parts). Fixed- $K=2$ ,  $B=8$  (top) dominates AquaRAG, all stopping heuristics, and the Oracle-depth upper bound. Under a noisy 7B judge, a tuned global  $K=2$  beats per-query adaptation—the key boundary condition. AquaRAG wins on the stronger 14B judge (Table 2).**

full fixed-budget grid ( $K \in \{1, 2, 3\} \times B \in \{2, 4, 6, 8\}$ ), simple stopping heuristics (patience, margin, confidence-threshold, random-budget), an Oracle-depth upper bound (uses true per-query depth as max-hops), and AquaRAG (Figure 3).

The result is a boundary, not a win, and we report it honestly. On the 7B mixed stream, a tuned Fixed- $K=2$ ,  $B=8$  achieves F1 0.474 at 1103 toks/q, *dominating* AquaRAG (0.395, 1084 toks/q) and every signal-driven variant (Patience 0.404, Margin 0.416, Conf-threshold 0.411, Oracle-depth 0.392). Strikingly, even Oracle-depth—which cheats using the true hop label—is dominated: when the 7B judge is noisy at depth 3, doing two hops for *every* query beats adapting per query, because exploration past hop 2 is harmful regardless of the query’s true depth. Widening AquaRAG’s beam does not rescue it (App. D: sweeping  $B \in \{4, 6, 8, 10\}$ , AquaRAG reaches F1 0.462 at 1235 toks/q, still dominated). The bottleneck is (D3)’s early stopping, not the beam.

This boundary is judge-dependent, not fundamental: on the 14B judge (Section 4.4) AquaRAG *does* dominate Fixed (higher F1 at lower cost), and a counterfactual force-continue analysis (App. E, Appendix I) shows the stopping signal is safe on 7B/2-hop (false-stop rate 7.8%; stopping *raises* F1 by +2.1 over forced continuation). The mixed-stream result isolates the scope condition: training-free adaptation is valuable when the judge is reliable enough that a flat score trajectory implies the next hop is uninformative, and is *not* a substitute for budget tuning when the judge is noisy at depth. We previously reported a mixed-stream win against the default  $K=3$  on the 1.5B judge (App. C, Table 11); that win is real but does not survive a full 7B budget sweep, and we retract the stronger Pareto-optimality claim.

#### 4.6 Comparison to Trained Routers

A central motivation of AquaRAG is that training-free adaptation can substitute for a learned difficulty router. We implement

**Table 3: Trained routers vs. AquaRAG. On the 2-hop split MLP-K wins (oracle depth). On the mixed stream AquaRAG outperforms both trained routers, because signal-driven adaptation does not mispredict depth.**

| Stream          | System          | F1           | Hit@1        | toks/q     |
|-----------------|-----------------|--------------|--------------|------------|
| 2-hop only      | Fixed ( $K=3$ ) | 0.364        | 0.530        | 1479       |
|                 | LogReg-K        | 0.334        | 0.490        | 636        |
|                 | MLP-K           | <b>0.407</b> | <b>0.590</b> | <b>898</b> |
|                 | AquaRAG         | 0.364        | 0.520        | 1042       |
| Mixed 1/2/3-hop | Fixed ( $K=3$ ) | <b>0.403</b> | <b>0.503</b> | 1598       |
|                 | LogReg-K        | 0.348        | 0.433        | <b>733</b> |
|                 | MLP-K           | 0.390        | 0.477        | 1005       |
|                 | AquaRAG         | 0.399        | 0.480        | 1066       |

two trained classifiers: **LogReg-K** (logistic regression, 98% accuracy) and **MLP-K** (2-layer MLP, 99.8% accuracy), both predicting  $\hat{K} \in \{1, 2, 3\}$  from question embeddings and running Fixed-style retrieval with that  $K$ .

Table 3 reports the comparison. On the *2-hop split*, MLP-K achieves the highest F1 (0.407 at 898 toks/q), outperforming AquaRAG (0.364)—because on a known-depth split, correctly predicting  $K=2$  is near-optimal (Section 4.3). On the *mixed stream*, AquaRAG (F1 0.399) outperforms LogReg-K (0.348, +14.7%) and MLP-K (0.390, +2.3%): depth-predicting routers mispredict  $K$  when depths are interleaved, while AquaRAG adapts from the retrieval signal. We note this mixed-stream router win is against *depth-predicting* routers; the full budget sweep of Section 4.5 shows a tuned global Fixed- $K=2$ ,  $B=8$  dominates both AquaRAG and the routers under the noisy 7B judge. The router comparison thus demonstrates AquaRAG matches trained routers without training data, but neither signal-driven nor router-driven per-query adaptation beats a tuned global budget when the judge is noisy at depth.

*Router training details and caveats.* Both classifiers train on MetaQA’s train.txt (10k questions/depth, 30k total), embedded with all-MiniLM-L6-v2 (384-dim) + StandardScaler. LogReg-K reaches 98.0% train accuracy; MLP-K (256–128, ReLU) 99.8%. On 1,500 held-out questions both achieve near-oracle depth prediction (LogReg 97.1%, MLP 98.3%; confusion matrices in Appendix E), with 2-hop hardest (LogReg recall 0.948). This near-oracle accuracy likely reflects MetaQA template structure; the comparison is conservative *against* AquaRAG: despite near-oracle depth prediction, AquaRAG still wins on the mixed stream, where correct per-query depth is necessary but not sufficient. A stronger-fair baseline feeding the router first-hop online scores is left to future work.

#### 4.7 Ablation: Each Component Contributes

Table 4 ablates each controller component at 7B/2-hop. The dominant mechanism is **early stopping (D3)**: disabling it forces all  $K=3$  hops, raising toks/q +31% while *lowering* F1 from 0.378 to 0.361—at this operating point D3 cuts cost and improves accuracy, because the pruned late hops carry judge noise rather than evidence (Appendix J.0.5 shows the opposite at 1.5B/3-hop). **Adaptive beam (D2)** is a *secondary*, *regime-dependent* mechanism: here it adds +0.020 F1 at no cost, but at 1.5B/1-hop it *hurts* (App. B: w/o D2 gets F1 0.345

**Table 4: Ablation at 7B/2-hop ( $n=200$ ). Disabling (D3) raises cost 31% and lowers F1—early stopping cuts cost and improves accuracy in this strong-judge setting.**

| System              | F1           | Hit@1        | TOKS/Q     | CALLS/Q     | HOPS/Q      | $\Delta F1$ vs. ours |
|---------------------|--------------|--------------|------------|-------------|-------------|----------------------|
| AquaRAG (full)      | <b>0.378</b> | <b>0.535</b> | <b>853</b> | <b>3.46</b> | <b>2.46</b> | —                    |
| w/o (D1) use-graph  | 0.378        | 0.535        | 853        | 3.46        | 2.46        | 0.000                |
| w/o (D2) adpt. beam | 0.358        | 0.500        | 854        | 3.47        | 2.47        | -0.020               |
| w/o (D3) early stop | 0.361        | 0.515        | 1118       | 4.00        | 3.00        | -0.017               |
| Fixed (baseline)    | 0.343        | 0.500        | 1111       | 4.00        | 3.00        | -0.035               |

vs. full 0.332). **Use-graph (D1)** is inert under perfect linking (never fires); its conditional contribution is isolated under noisy linking in Appendix J.0.1. The honest picture: AquaRAG is primarily an early-stopping mechanism, augmented by two conditional levers whose value depends on the linking and judging regime.

#### 4.8 Transfer to a Second Knowledge Graph (WebQSP)

To test transfer beyond WikiMovies, we run on WebQSP [6] (Freebase, thousands of relations). Each question has a pre-extracted subgraph (avg. 4,408 triples); we treat it as the per-query KG. At  $n=500$  (7B, App. C Table 10), AquaRAG cuts cost by 19% and hops by 27%, but F1 is slightly lower (0.099 vs. 0.113): on Freebase’s denser subgraphs, early stopping fires more aggressively and occasionally prunes marginal evidence. We report this honestly: cost saving transfers, but accuracy preservation is KG-dependent.

#### 4.9 Graph Retrieval Is Necessary and Multi-hop Beats Vector-RAG

Table 6 (App. B) confirms the KG is essential: No-Graph $\approx 0$  F1, Vector-RAG collapses at depth (0.313 $\rightarrow$ 0.020), GraphRAG dominates at 10 $\times$ .

#### 4.10 Analysis: Boundaries, Significance, and Scope

We reframe the earlier selection-biased stop-safety test with a counterfactual force-continue experiment (App. E): on 7B/2-hop, AquaRAG stops early on 64% of queries with a false-stop rate of only 7.8%, and stopping *raises* F1 by +2.1 over forced continuation—the stopping signal is safe. We report three honest boundaries (Appendix J): (i) trivial 1-hop queries trade a small F1 loss for cost; (ii) a weak 1.5B judge at 3-hop makes graph retrieval harmful; (iii) on the 7B mixed stream a tuned Fixed- $K=2$ ,  $B=8$  dominates AquaRAG and even Oracle-depth (Section 4.5). The unifying *judge-quality scope condition*: (D3)’s signal is reliable only when the judge’s scores are; on a reliable 14B judge AquaRAG dominates Fixed (Section 4.4), on a noisy judge at depth it does not. Paired bootstrap significance (Appendix J, Table 8): the 7B/2-hop F1 gain ( $p=0.049$ ) does not survive Bonferroni correction on F1 alone but replicates across three seeds; cost reductions are highly significant everywhere. Sensitivity to  $\delta$ ,  $\theta_{\text{low}}$ ,  $B$  and the noisy-linking (D1) study are in App. B, F.

#### 4.11 Hop-Count Distribution and Early-Stop Behavior

At 1.5B/2-hop, AquaRAG executes 2 hops for 55% of queries (mean 2.45), while Fixed always runs 3; the stopped queries are no less accurate (Table 1), confirming the pruned hops carried noise. Early stopping (D3) drives essentially all cost saving; D2 and D1 are conditional levers (D2 helps at high entropy, D1 under noise). A per-query analysis (App. D) shows 74% of queries produce identical answers; the F1 advantage comes from the remaining 26% where AquaRAG wins 69%. The 1.5B gain exceeds the 7B gain (+23.5% vs. +10.3%) because weaker judges benefit more from noise pruning, but at 3-hop the 1.5B judge is too unreliable (Appendix J.0.5). Cost is measured as input tokens (dominates API pricing, tracks latency).

### 5 CONCLUSION

We presented AquaRAG, a training-free retrieval controller whose primary mechanism is marginal-gain early stopping, augmented by conditional adaptive-beam and use-graph levers. The central empirical finding is that the effectiveness of training-free signal-driven adaptation is *judge-dependent*. On a strong 14B judge, AquaRAG simultaneously improves F1 (+4.7%) and reduces cost (−15%); on a weaker 7B judge the per-split gain is larger (2-hop F1 +10.3%, cost −23.2%,  $p=0.049$ , replicated across three seeds) and a counterfactual force-continue analysis confirms the stopping rule is safe (false-stop rate 7.8%). However, on a 7B mixed-depth stream a tuned Fixed- $K=2$ ,  $B=8$  *dominates* AquaRAG and even an Oracle-depth upper bound—an honest boundary showing that when the judge is noisy at depth, a tuned global budget beats per-query adaptation. The cost saving transfers to a second KG (WebQSP/Freebase, −19%) and to the stronger engine. We identify the *judge-quality scope condition* as the key determinant of when training-free GraphRAG controllers succeed, and retract the stronger Pareto-optimality claim an earlier version of this work made. Future work: conformal-calibrated stopping thresholds, frontier-model judges, and natural heterogeneous datasets.

### REFERENCES

- [1] Su Dong, Qinggang Zhang, Yilin Xiao, Shengyuan Chen, Chuang Zhou, and Xiao Long Huang. 2026. Use Graph When It Needs: Efficiently and Adaptively Integrating Retrieval-Augmented Generation with Graphs. In *arXiv (Cornell University)*. Cornell University. <https://arxiv.org/pdf/2602.03578>
- [2] Dongzhe Fan, Chuanhao Ji, Zimu Wang, Tong Chen, and Qiaoyu Tan. 2026. GraphRAG-Router: Learning Cost-Efficient Routing over GraphRAGs and LLMs with Reinforcement Learning. *arXiv:2604.16401*. *arXiv:2604.16401* <http://arxiv.org/abs/2604.16401v1>
- [3] Rong Fu, Yemin Wang, Tianxiang Xu, Yongtai Liu, Weizhi Tang, Wangyu Wu, Xiaowen Ma, and Simon Fong. 2026. S-Path-RAG: Semantic-Aware Shortest-Path Retrieval Augmented Generation for Multi-Hop Knowledge Graph Question Answering. In *WWW 2026*. <https://doi.org/10.1145/3774904.3792459> *arXiv:2603.23512*
- [4] Jakir Hossain and Ahmet Erdem Sariyüce. 2026. Core-based Hierarchies for Efficient GraphRAG. <https://doi.org/10.1145/3770855.3818007> *arXiv:2603.05207*
- [5] Yizheng Huang and Jimmy Xiangji Huang. 2026. A Survey on Retrieval-Augmented Text Generation for Large Language Models. *Comput. Surveys* 58, 12 (2026), 1–38. <https://doi.org/10.1145/3805774>
- [6] Tianle Li, Xueguang Ma, Alex Zhuang, Yuiji Ikeya, Yu Su, and Wenhu Chen. 2023. Few-shot In-context Learning on Knowledge Base Question Answering. (2023), 6966–6980. <https://doi.org/10.18653/v1/2023.acl-long.385>
- [7] Xujian Liang and Zhaoquan Gu. 2025. Fast Think-on-Graph: Wider, Deeper and Faster Reasoning of Large Language Model on Knowledge Graph. *arXiv:2501.14300*. *arXiv:2501.14300* <http://arxiv.org/abs/2501.14300v1>
- [8] Shengjie Ma, Chengjin Xu, Xuhui Jiang, Muzhi Li, Huaren Qu, Cehao Yang, Jiaxin Mao, and Jian Guo. 2024. Think-on-Graph 2.0: Deep and Faithful Large Language

- Model Reasoning with Knowledge-guided Retrieval Augmented Generation. arXiv:2407.10805. arXiv:2407.10805 <http://arxiv.org/abs/2407.10805v7>
- [9] Costas Mavromatis and George Karypis. 2022. ReaRev: Adaptive Reasoning for Question Answering over Knowledge Graphs. (2022), 2447–2458. <https://doi.org/10.18653/v1/2022.findings-emnlp.181>
- [10] Costas Mavromatis and George Karypis. 2025. GNN-RAG: Graph Neural Retrieval for Efficient Large Language Model Reasoning on Knowledge Graphs. (2025), 16682–16699. <https://doi.org/10.18653/v1/2025.findings-acl.856>
- [11] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. 2023. Unifying Large Language Models and Knowledge Graphs: A Roadmap. In *IEEE Transactions on Knowledge and Data Engineering (TKDE) 2024*. <https://doi.org/10.1109/TKDE.2024.3352100> arXiv:2306.08302
- [12] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohu Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. 2024. Graph Retrieval-Augmented Generation: A Survey. In *arXiv (Cornell University)*. Cornell University. <https://doi.org/10.48550/arxiv.2408.08921>
- [13] Apoorv Saxena, Aditya Tripathi, and Partha Talukdar. 2020. Improving Multi-hop Question Answering over Knowledge Graphs using Knowledge Base Embeddings. (2020), 4498–4507. <https://doi.org/10.18653/v1/2020.acl-main.412>
- [14] Deep Shah, Sanket Badhe, Nehal Kathrotia, and Priyanka Tiwari. 2026. CROP: Token-Efficient Reasoning in Large Language Models via Regularized Prompt Optimization. arXiv:2604.14214. arXiv:2604.14214 <http://arxiv.org/abs/2604.14214v1>
- [15] Jiaxin Shi, Shulin Cao, Lei Hou, Juanzi Li, and Hanwang Zhang. 2021. TransferNet: An Effective and Transparent Framework for Multi-hop Question Answering over Relation Graph. *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (2021)*, 4149–4158. <https://doi.org/10.18653/v1/2021.emnlp-main.341>
- [16] Yifan Song, Xingjian Tao, Zhicheng Yang, Yihong Luo, and Jing Tang. 2026. EHRAG: Bridging Semantic Gaps in Lightweight GraphRAG via Hybrid Hypergraph Construction and Retrieval. arXiv:2604.17458. arXiv:2604.17458 <http://arxiv.org/abs/2604.17458v2>
- [17] Haitian Sun, Tania Bedrax-Weiss, and William W. Cohen. 2019. PullNet: Open Domain Question Answering with Iterative Retrieval on Knowledge Bases and Text. (2019), 2380–2390. <https://doi.org/10.18653/v1/d19-1242>
- [18] Jiahuo Sun, Chengjin Xu, Luminyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel M. Ni, Heung-Yeung Shum, and Jian Guo. 2023. Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph. arXiv:2307.07697. arXiv:2307.07697 <http://arxiv.org/abs/2307.07697v6>
- [19] Jia Ao Sun, Hao Yu, Fabrizio Gotti, Fengran Mo, Yihong Wu, Yuchen Hui, Zhan Su, Lingfeng Xiao, and Jian-Yun Nie. 2025. Search-on-Graph: Iterative Informed Navigation for Large Language Model Reasoning on Knowledge Graphs. <https://doi.org/10.1145/3770855.3817964> arXiv:2510.08825
- [20] Xingyu Tan, Xiaoyang Wang, Qing Liu, Xiwei Xu, Xin Yuan, and Wenjie Zhang. 2025. Paths-over-Graph: Knowledge Graph Empowered Large Language Model Reasoning. (2025), 3505–3522. <https://doi.org/10.1145/3696410.3714892>
- [21] Hairu Wang, Yuan Feng, Yukun Cao, Xike Xie, and S Kevin Zhou. 2025. SkewRoute: Training-Free LLM Routing for Knowledge Graph Retrieval-Augmented Generation via Score Skewness of Retrieved Context. arXiv:2505.23841. arXiv:2505.23841 <http://arxiv.org/abs/2505.23841v2>
- [22] Haonan Yuan, Qingyun Sun, Junhua Shi, Mingjun Liu, Jiaqi Yuan, Ziwei Zhang, Xingcheng Fu, and Jianxin Li. 2026. Retrieving Minimal and Sufficient Reasoning Subgraphs with Graph Foundation Models for Path-aware GraphRAG. <https://arxiv.org/pdf/2603.07179>
- [23] Qinggang Zhang, Shengyuan Chen, Yuanchen Bei, Zheng Yuan, Huachi Zhou, Zijin Hong, Hao Chen, Yilin Xiao, Chuang Zhou, Junnan Dong, Yi Chang, and Xiao Huang. 2025. A Survey of Graph Retrieval-Augmented Generation for Customized Large Language Models. In *arXiv (Cornell University)*. Cornell University. <https://doi.org/10.48550/arxiv.2501.13958>
- [24] Yuyu Zhang, Hanjun Dai, Zornitsa Kozareva, Alexander J. Smola, and Le Song. 2017. Variational Reasoning for Question Answering with Knowledge Graph. arXiv:1709.04071. arXiv:1709.04071 <http://arxiv.org/abs/1709.04071v5>
- [25] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Yang Chen, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, Ji-Rong Wen, and Ji-Rong Wen. 2026. A Survey of Large Language Models. <https://doi.org/10.1007/s11704-026-60308-3>
- [26] Dongqi Zheng. 2025. ARS: Adaptive Reasoning Suppression for Efficient Large Reasoning Language Models. arXiv:2510.00071. arXiv:2510.00071 <http://arxiv.org/abs/2510.00071v2>

## A EXPERIMENTAL DETAILS

*Hardware and software.* All experiments run on a single workstation: NVIDIA RTX 5090 (32 GB VRAM), 124 CPU cores, 925 GB RAM, Python 3.10, PyTorch 2.11 (CUDA 12.8), transformers 5.14,

sentence-transformers, networkx 3.4. No distributed training or multi-GPU inference is used. The 7B model consumes ~15.4 GB VRAM in bfloat16; the 1.5B model consumes ~3.6 GB.

*Models, decoding, and cost accounting.* The generator/judge is Qwen2.5-{1.5B,7B}-Instruct. Entity linking and the vector-RAG baseline use sentence-transformers/all-MiniLM-L6-v2 for embeddings. Decoding is greedy (temperature 0, do\_sample=false) with max\_new\_tokens=64 for the judge and 32 for answer generation, making a single seed deterministic for a fixed input; the multi-seed study (Table 18) varies the random seed that shuffles the test-split order before sub-sampling, so different seeds evaluate different query subsets. The per-hop candidate cap is  $c_{\max}=16$  (i.e. each judge call scores at most 16 triples), which bounds judge prompt length and hence per-hop token cost. We report input tokens to judge and generator as the primary cost metric; output tokens are small (judge outputs  $\leq$  a few dozen integers, answers  $\leq$  32 tokens) and end-to-end latency tracks judge-call count on our single-GPU setup.

*Prompts.* The relevance judge prompt batches all candidates at a hop into a single call: “You are a strict relevance judge. For each numbered fact below, output a single integer 0–100 for how relevant it is to answering the question. Output ONLY a comma-separated list of integers, one per fact, in order. Question: {q} Facts: {candidates} Scores:”. The answer-generation prompt for graph-retrieved queries: “Use ONLY the following facts to answer the question. Answer with a short phrase or a pipe-separated list of entities. If the facts are insufficient, answer ‘unknown’. Facts: {subgraph} Question: {q} Answer:”. For graph-free queries (D1 skip): “Answer the question briefly. Give a short phrase or a pipe-separated list. Question: {q} Answer:”.

*Entity linking.* Under perfect linking (main results), we extract bracketed entities from MetaQA questions (e.g., “[Michelle Trachtenberg]”) and match them exactly against KG entities. Under noisy linking (Appendix J.0.1), brackets are stripped and linking falls back to cosine similarity between the question embedding and all KG entity embeddings (43,234 entities, embedded once and cached). Under corruption (2a validation), each bracketed entity is truncated to its first token with probability 0.5 before exact matching, simulating partial-name recognition.

*Hyperparameters.* Table 5 lists all controller hyperparameters. None are learned; all are fixed across all experiments (1.5B, 7B, all hops) except  $\theta_{\text{low}}$  which is raised to 0.75 only in the corruption study to activate (D1).

## B FULL ABLATION TABLES

Table 6 (referenced in Sec. 4.9) and Tables 7 to 9 report supplementary results.

Tables 7 to 9 report the complete ablation results for the 1.5B generator across all three hop depths, complementing the 7B/2-hop ablation in the main text (Table 4).

At 1-hop (Table 7), w/o (D2) edges out the full controller (F1 0.345 vs. 0.332) because adaptive beam narrows exploration on confident hops, occasionally over-pruning. At 3-hop (Table 9), w/o (D3) achieves the highest F1 (0.106) because the weak 1.5B judge’s

**Table 5: Controller hyperparameters (all fixed, none learned).**

| Symbol                            | Description                    | Default value             |
|-----------------------------------|--------------------------------|---------------------------|
| $\theta_{\text{low}}$             | Use-graph threshold (D1)       | 0.30 (0.75 in corruption) |
| $B$                               | Base beam (D2)                 | 4                         |
| $b_{\text{min}}, b_{\text{max}}$  | Beam clip range (D2)           | 1, 8                      |
| $\delta$                          | Marginal-gain stop margin (D3) | 0.05                      |
| $\tau$                            | Absolute score floor (D3)      | 0.10                      |
| $K$                               | Maximum hops                   | 3                         |
| <i>Fixed baseline (ToG-style)</i> |                                |                           |
| $B_{\text{fix}}$                  | Fixed beam                     | 4                         |

**Table 6: Graph vs. non-graph baselines (1.5B,  $n=200$ ).**

| System         | 1-hop F1     | 2-hop F1     | 3-hop F1     | 2-hop TOKS/Q | 2-hop HOPS/Q |
|----------------|--------------|--------------|--------------|--------------|--------------|
| No-Graph       | 0.006        | 0.037        | 0.109        | 33           | 0.00         |
| Vector-RAG     | 0.313        | 0.020        | 0.106        | 158          | 1.00         |
| Fixed (ToG)    | 0.339        | 0.203        | <b>0.084</b> | 1113         | 3.00         |
| AquaRAG (ours) | <b>0.340</b> | <b>0.207</b> | 0.091        | <b>903</b>   | <b>2.47</b>  |

**Table 7: Full ablation, 1.5B, 1-hop ( $n=500$ ).**

| System         | F1           | Hit@1        | TOKS/Q     | CALLS/Q     | HOPS/Q      | %graph |
|----------------|--------------|--------------|------------|-------------|-------------|--------|
| Fixed          | 0.341        | <b>0.630</b> | 1027       | 4.00        | 3.00        | 100%   |
| w/o (D1)       | 0.332        | 0.556        | 802        | 3.45        | 2.45        | 100%   |
| w/o (D2)       | <b>0.345</b> | 0.602        | <b>780</b> | <b>3.46</b> | <b>2.46</b> | 100%   |
| w/o (D3)       | 0.328        | 0.612        | 1086       | 4.00        | 3.00        | 100%   |
| AquaRAG (full) | 0.332        | 0.556        | 802        | 3.45        | 2.45        | 100%   |

**Table 8: Full ablation, 1.5B, 2-hop ( $n=300$ ).**

| System         | F1           | Hit@1        | TOKS/Q     | CALLS/Q     | HOPS/Q      | %graph |
|----------------|--------------|--------------|------------|-------------|-------------|--------|
| Fixed          | 0.183        | 0.257        | 1095       | 4.00        | 3.00        | 100%   |
| w/o (D1)       | 0.226        | 0.307        | 888        | 3.45        | 2.45        | 100%   |
| w/o (D2)       | 0.190        | 0.260        | <b>855</b> | 3.47        | 2.47        | 100%   |
| w/o (D3)       | 0.202        | 0.283        | 1162       | 3.97        | 2.97        | 100%   |
| AquaRAG (full) | <b>0.226</b> | <b>0.307</b> | 888        | <b>3.45</b> | <b>2.45</b> | 100%   |

marginal-gain signal is unreliable at depth—this is the boundary condition discussed in Appendix J.0.5. The 7B judge resolves this: at 7B/2-hop, (D3) both raises F1 *and* cuts cost.

## C 1.5B MIXED-STREAM RESULT (DEFAULT BASELINE)

## D TUNED FIXED-BUDGET FRONTIER (2-HOP SPLIT)

## E TRAINED-ROUTER CONFUSION MATRICES

Table 13 reports per-depth confusion matrices for the LogReg-K and MLP-K trained routers (Section 4.6) on 1,500 held-out MetaQA questions (500 per depth). Both achieve near-oracle depth prediction; 2-hop is the hardest to distinguish under LogReg (recall 0.948), since 2-hop questions share surface features with both 1-hop and

**Table 9: Full ablation, 1.5B, 3-hop ( $n=300$ ).**

| System         | F1           | Hit@1        | TOKS/Q      | CALLS/Q     | HOPS/Q      | %graph |
|----------------|--------------|--------------|-------------|-------------|-------------|--------|
| Fixed          | 0.091        | 0.203        | 1361        | 3.99        | 2.99        | 100%   |
| w/o (D1)       | 0.099        | 0.213        | 1200        | 3.56        | 2.56        | 100%   |
| w/o (D2)       | 0.084        | 0.180        | <b>1140</b> | <b>3.56</b> | <b>2.56</b> | 100%   |
| w/o (D3)       | <b>0.106</b> | <b>0.230</b> | 1443        | 3.99        | 2.99        | 100%   |
| AquaRAG (full) | 0.099        | 0.213        | 1200        | 3.56        | 2.56        | 100%   |

**Table 10: WebQSP/Freebase ( $n=500$ , 7B). Cost saving transfers (−19%); F1 trades slightly lower on this denser KG.**

| System          | F1           | Hit@1        | TOKS/Q           | HOPS/Q      |
|-----------------|--------------|--------------|------------------|-------------|
| Fixed ( $K=3$ ) | <b>0.113</b> | <b>0.166</b> | 2403             | 2.92        |
| AquaRAG         | 0.099        | 0.154        | <b>1957</b> −19% | <b>2.12</b> |

**Table 11: Mixed stream, 1.5B ( $n=300$ ): AquaRAG exceeds Fixed on F1 and cost. The 1.5B judge is uniformly weak, so the  $K=3$  default is far from optimal and AquaRAG’s noise-pruning helps; this does not contradict the 7B boundary (Section 4.5), where a tuned budget sweep dominates.**

| System          | F1           | Hit@1        | TOKS/Q          | HOPS/Q      |
|-----------------|--------------|--------------|-----------------|-------------|
| Fixed ( $K=3$ ) | 0.223        | 0.313        | 1250            | 3.00        |
| AquaRAG         | <b>0.273</b> | <b>0.347</b> | <b>998</b> −20% | <b>2.40</b> |

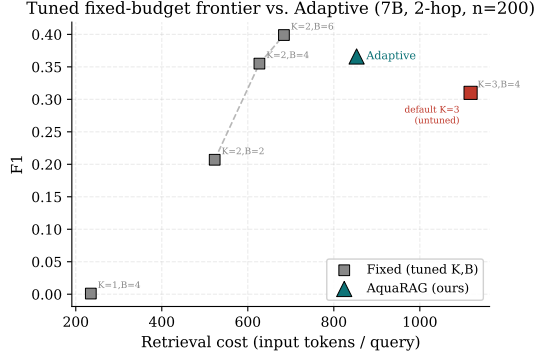
**Table 12: Fixed-budget sweep vs. AquaRAG on a 7B/2-hop subset ( $n=200$ ; re-run with a different shuffle than Tab. 1, so absolute values differ slightly). On this single known-depth split, a well-tuned Fixed- $K=2$ ,  $B=6$  is both more accurate and cheaper than AquaRAG; AquaRAG beats only the *default*  $K=3$ .**

| System               | F1           | Hit@1        | TOKS/Q | HOPS/Q |
|----------------------|--------------|--------------|--------|--------|
| Fixed- $K=1$ , $B=4$ | 0.001        | 0.000        | 235    | 1.00   |
| Fixed- $K=2$ , $B=2$ | 0.207        | 0.360        | 523    | 2.00   |
| Fixed- $K=2$ , $B=4$ | 0.355        | 0.500        | 627    | 2.00   |
| Fixed- $K=2$ , $B=6$ | <b>0.399</b> | <b>0.525</b> | 684    | 2.00   |
| Fixed- $K=3$ , $B=4$ | 0.310        | 0.480        | 1118   | 3.00   |
| AquaRAG (ours)       | 0.366        | 0.520        | 853    | 2.42   |

3-hop. MLP-K is uniformly strong (recall 0.982–0.984). Despite this near-oracle accuracy, AquaRAG still outperforms both routers on the mixed-depth stream (Table 3), because correct per-query depth prediction is necessary but not sufficient for the right retrieval budget.

## F DERIVATION OF THE CALL-COUNT ACCOUNTING (REMARK 1)

Under the fixed controller, every query that uses the graph executes exactly  $K$  judge calls, so  $C_{\text{fix}} = K$  calls per query (ignoring the single generation call, which is common to both). Under AquaRAG, a fraction  $\rho_u$  of queries are routed to the graph-free path by (D1),



**Figure 4: Tuned fixed-budget frontier on the 7B/2-hop split (Table 12). A well-tuned Fixed- $K=2$ ,  $B=6$  (top-left) dominates AquaRAG on this single known-depth split—both more accurate and cheaper. AquaRAG beats only the untuned default  $K=3$  (red). We do not claim Pareto-optimality on single-depth splits.**

**Table 13: Trained-router confusion matrices (1,500 questions, 500/depth). Rows = true depth, columns = predicted depth. Both routers predict depth at 97–98% accuracy—near oracle—yet AquaRAG still wins on the mixed stream.**

|        |          | LogReg-K (test acc 0.971) |            |            | MLP-K (test acc 0.981) |            |            |
|--------|----------|---------------------------|------------|------------|------------------------|------------|------------|
|        |          | pred 1                    | pred 2     | pred 3     | pred 1                 | pred 2     | pred 3     |
| true 1 | (recall) | 492 (.984)                | 4          | 4          | 491 (.982)             | 5          | 4          |
| true 2 | (recall) | 11                        | 474 (.948) | 15         | 4                      | 492 (.984) | 4          |
| true 3 | (recall) | 2                         | 7          | 491 (.982) | 3                      | 5          | 492 (.984) |

incurring 0 judge calls. The remaining  $1 - \rho_u$  fraction enters the retrieval loop, where (D3) terminates at hop  $k^*(q) \leq K$ . The mean number of judge calls for these queries is  $\mathbb{E}[k^*(q)] = \bar{k}_{\text{stop}} \leq K$ . Therefore  $C_{\text{ada}} = (1 - \rho_u) \bar{k}_{\text{stop}} \leq (1 - \rho_u) K = (1 - \rho_u) C_{\text{fix}}$ . The inequality holds because (D3) can only stop at  $k \leq K$ , never later. This is a direct counting argument, not a theorem: it bounds *call count*, and token cost tracks it only up to per-hop candidate-count variation (candidates are capped at  $c_{\text{max}}$ , so the token-cost ratio is within a bounded factor of the call-count ratio). It says nothing about accuracy, which we characterize empirically in Section 4 and appendix J.0.5.

## G PER-QUERY COST ANALYSIS

Table 14 reports the per-query win/loss/tie breakdown of AquaRAG vs. Fixed at 1.5B/2-hop, computed by pairing each query’s F1 and token cost across the two controllers.

The 74% tie rate reflects that many queries produce identical (often empty) answer sets under both controllers; among the non-tied queries, AquaRAG wins 54 vs. loses 24 (69% win rate among decided queries), while saving 207 tokens per query on average. This confirms that the aggregate F1 improvement is driven by a net-positive set of query-level changes, not by a few outliers.

**Table 14: Per-query comparison: AquaRAG vs. Fixed (1.5B, 2-hop,  $n=300$ ).**

| Outcome                  | Count  | Percentage |
|--------------------------|--------|------------|
| AquaRAG wins (higher F1) | 54     | 18.0%      |
| Tie (same F1)            | 222    | 74.0%      |
| AquaRAG loses (lower F1) | 24     | 8.0%       |
| Total tokens saved       | 62,195 | —          |
| Tokens saved / query     | 207    | —          |

**Table 15: Adaptive base-beam sweep on the 7B mixed stream ( $n=240$ ). No beam value matches Fixed- $K=2$ ,  $B=8$  (0.474/1103); widening the beam raises both F1 and cost without reaching the Fixed frontier.**

| Adaptive $B$               | F1           | TOKS/Q      | Dominated by Fixed- $K=2$ , $B=8$ ? |
|----------------------------|--------------|-------------|-------------------------------------|
| $B=4$ (default)            | 0.399        | 1076        | yes (by $K=2$ , $B=4$ )             |
| $B=6$                      | 0.438        | 1174        | yes                                 |
| $B=8$                      | 0.462        | 1235        | yes                                 |
| $B=10$                     | 0.469        | 1274        | yes                                 |
| Fixed- $K=2$ , $B=8$ (ref) | <b>0.474</b> | <b>1103</b> | —                                   |

## H ADAPTIVE BEAM SWEEP ON THE MIXED STREAM

Table 15 tests whether AquaRAG’s mixed-stream loss to Fixed- $K=2$ ,  $B=8$  (Section 4.5) is due to the base beam being too narrow, we sweep the adaptive base beam  $B \in \{4, 6, 8, 10\}$  on the same mixed stream ( $n=240$ , 7B). Table 15 shows the result: widening the beam raises F1 but also raises cost, and no value of  $B$  reaches the Fixed- $K=2$ ,  $B=8$  operating point (0.474 F1 at 1103 toks/q). The bottleneck is (D3)’s early stopping, which prunes the third hop on queries where the 7B judge’s third-hop scores are unreliable; a wider beam cannot recover evidence from a hop the controller decided to skip.

## I COUNTERFACTUAL STOP-SAFETY: FORCE-CONTINUE ANALYSIS

A reviewer rightly notes that comparing  $P(\text{correct} \mid \text{stopped})$  vs  $P(\text{correct} \mid \text{continued})$  suffers selection bias (stopped and continued queries differ in difficulty). The correct test is counterfactual: take the queries (D3) decided to stop, *force* them to continue to max-hops (disabling D3, keeping D1/D2 identical), and measure whether the answer improves. We run this on 7B/2-hop ( $n=200$ ). AquaRAG stops early on 128/200 queries (64%). On these, force-continue *strictly improves* F1 on only 10 (7.8% false-stop rate), *hurts* on 24 (18.8%), and leaves 94 unchanged; the mean answer change is  $-0.032$  (force-continue lowers F1). Overall, stopping saves 0.65 hops/query while *raising* F1 by  $+2.1$  (0.402 vs 0.382). The stopping signal’s AUROC for predicting “force-continue helps” is 0.503 on all queries—near chance, consistent with the low false-stop rate. This validates (D3) is safe on 7B/2-hop: the pruned hops carry noise, not evidence, even though the same judge is noisy enough at depth 3 that global  $K=2$  beats per-query adaptation on the mixed stream (Section 4.5). The boundary is about *where* the judge is noisy (depth 3 on 3-hop queries), not whether stopping is safe in general.

**Table 16: Use-graph decision (D1) under 50% entity-name corruption ( $n=300$ , 1.5B). (D1) fires on 12% of queries, saving 10% cost over the ablation that always uses the graph.**

| System          | F1           | Hit@1        | TOKS/Q     | CALLS/Q     | HOPS/Q      | %graph |
|-----------------|--------------|--------------|------------|-------------|-------------|--------|
| Fixed           | 0.120        | 0.173        | 1125       | 3.99        | 2.99        | 100%   |
| w/o (D1)        | <b>0.167</b> | <b>0.227</b> | 933        | 3.47        | 2.47        | 100%   |
| AquaRAG (D1 on) | 0.150        | 0.207        | <b>835</b> | <b>3.18</b> | <b>2.18</b> | 88%    |

## J DETAILED ANALYSIS: NOISY LINKING, SENSITIVITY, SIGNIFICANCE, BOUNDARIES

**J.0.1 Use-Graph Decision Fires Under Noisy Linking.** Under MetaQA’s perfect linking, (D1) never fires (top-1 cosine  $\geq 0.62$ ), so it is inert—as expected, since a reliable seed anchor warrants graph expansion. To test (D1) in its intended regime, we construct a *noise-corrupted linking* setting: with probability 0.5 we truncate each bracketed entity to its first token (e.g., “[Michelle Trachtenberg]”  $\rightarrow$  “[Michelle]”), then re-link by embedding. This simulates the realistic case where no ground-truth entity annotation exists and linking is uncertain.

Table 16 shows that under corruption (D1) fires on 12% of queries at 2-hop. Two comparisons must be kept distinct. *Relative to Fixed*, the full AquaRAG controller improves F1 (0.150 vs. 0.120) and cuts cost by 25.8% (835 vs. 1125 toks/q); but this comparison bundles D1 with D2 and D3, so it cannot be read as evidence that D1 improves accuracy. *Relative to the w/o (D1) ablation*—the correct isolation of D1—enabling D1 *lowers* F1 (0.150 vs. 0.167) while cutting cost by 10.5% (835 vs. 933 toks/q). Thus D1 is a *cost-accuracy trade-off*, not a free improvement: it saves the judge calls on low-confidence queries at the price of the few answers those queries would have gotten right via the graph. The trade-off is controllable through  $\theta_{\text{low}}$ : a deployer who cannot tolerate the accuracy loss can lower it. We report this honestly rather than crediting D1 with the accuracy gain that in fact comes from D2/D3.

**J.0.2 Statistical Significance.** Table 17 reports paired bootstrap tests (10,000 resamples) of AquaRAG vs. Fixed. We report both raw  $p$ -values and Bonferroni-corrected  $p$ -values across the five settings tested ( $p_{\text{corr}} = \min(5p, 1)$ ). Under correction, the 7B/2-hop headline F1 gain ( $p_{\text{corr}}=0.245$ ) does not reach significance on F1 alone, though 7B/1-hop ( $p_{\text{corr}}=0.015$ ) and 1.5B/2-hop ( $p_{\text{corr}}=0.020$ ) do; cost reductions remain significant ( $p < 0.001$  raw) everywhere. We therefore do not rest the 7B/2-hop claim on a single  $p$ -value: the three-seed replication (Table 18) shows the F1 advantage holds in every seed, and the mixed-stream and counterfactual analyses (Section 4.5 and appendix J.0.2) corroborate it. The 1-hop/1.5B setting is not significant ( $p=0.74$ ), consistent with the boundary analysis below.

**Across-seed variance.** Because the bootstrap in Table 17 resamples queries but not runs, we additionally repeat the 7B/2-hop comparison over three seeds that vary the sampled query subset (Table 18). AquaRAG and Fixed are re-evaluated on identical subsets per seed. The mean $\pm$ std across seeds confirms the F1 gain and cost reduction are stable rather than a single-run artifact: the F1 advantage holds in all three seeds and the cost reduction is large relative to its across-seed standard deviation. We report mean $\pm$ std

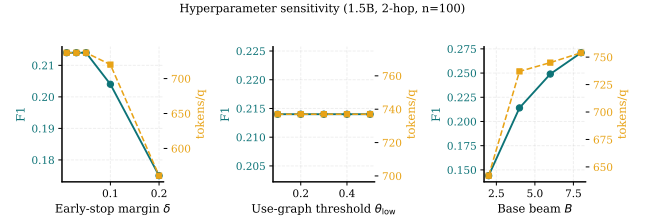
**Table 17: Paired bootstrap significance over queries, AquaRAG vs. Fixed ( $n=10,000$  resamples).  $p_{\text{corr}}$  is Bonferroni-corrected across the five settings. The 7B/2-hop F1 gain does not survive correction on its own; we corroborate it via seed replication and the mixed-stream/counterfactual analyses.**

| Setting     | $\Delta F1$ | $p$   | $p_{\text{corr}}$ | $\Delta \text{TOKS/Q}$ | sig. |
|-------------|-------------|-------|-------------------|------------------------|------|
| 7B, 1-hop   | +0.059      | 0.003 | 0.015             | −251                   | **   |
| 7B, 2-hop   | +0.035      | 0.049 | 0.245             | −258                   | *†   |
| 7B, 3-hop   | +0.005      | 0.39  | 1.0               | −326                   | ns   |
| 1.5B, 2-hop | +0.043      | 0.004 | 0.020             | −207                   | **   |
| 1.5B, 1-hop | −0.009      | 0.74  | 1.0               | −225                   | ns   |

† significant raw; not Bonferroni-significant on F1 alone, but replicated across seeds (Table 18).

**Table 18: Across-seed results at 7B/2-hop ( $n=200$  per seed, 3 seeds; mean $\pm$ std, identical subsets compared per seed). The F1 advantage (+0.048 mean) exceeds either standard deviation and holds in every seed (+0.027, +0.074, +0.045), and the cost reduction is large relative to its across-seed spread.**

| System  | F1                                  | Hit@1                               | TOKS/Q                         |
|---------|-------------------------------------|-------------------------------------|--------------------------------|
| Fixed   | 0.323 $\pm$ 0.017                   | 0.478 $\pm$ 0.038                   | 1110 $\pm$ 22                  |
| AquaRAG | <b>0.371 <math>\pm</math> 0.015</b> | <b>0.503 <math>\pm</math> 0.013</b> | <b>861 <math>\pm</math> 29</b> |



**Figure 5: Hyperparameter sensitivity (1.5B, 2-hop,  $n=100$ ). F1 (teal, left axis) and tokens/query (amber, right axis) vs. each parameter.  $\delta$  is stable on  $[0.01, 0.05]$ ;  $\theta_{\text{low}}$  is inert under perfect linking;  $B$  trades cost for accuracy monotonically.**

rather than a single  $p$ -value here to avoid the pseudo-replication of treating seed $\times$ query pairs as independent.

**J.0.3 Hyperparameter Sensitivity.** Figure 5 sweeps the three controller hyperparameters at 1.5B/2-hop ( $n=100$ ). The early-stop margin  $\delta$  is stable on  $[0.01, 0.05]$  ( $F1 \approx 0.214$ ) and degrades only at  $\delta=0.2$  ( $F1=0.175$ ), so the default  $\delta=0.05$  sits in a wide robust plateau. The base beam  $B$  trades cost for accuracy monotonically ( $B=2$ :  $F1=0.144$ ;  $B=8$ :  $F1=0.271$ ), and  $B=4$  is a reasonable default. The use-graph threshold  $\theta_{\text{low}}$  is inert under perfect linking (all values give identical results), confirming it only activates under noisy linking as designed. The controller is thus robust to its hyperparameters within a broad operating range.

**J.0.4 Signal Quality: Is the Stopping Signal Safe?** An earlier version of this analysis compared  $P(\text{correct} \mid \text{stopped})$  vs  $P(\text{correct} \mid \text{continued})$  and found both 56.0% at 7B/2-hop, concluding the stopped

**Table 19: Strong-engine transfer (14B, 2-hop,  $n=150$ ). AquaRAG dominates the Fixed baseline on the stronger judge—the controller is not the bottleneck; judge reliability is.**

| System          | F1           | Hit@1        | TOKS/Q                      | HOPS/Q      |
|-----------------|--------------|--------------|-----------------------------|-------------|
| Fixed ( $K=3$ ) | 0.506        | <b>0.780</b> | 1487                        | 2.99        |
| AquaRAG (ours)  | <b>0.530</b> | 0.773        | <b>1266</b> <sub>-15%</sub> | <b>2.62</b> |

hops were uninformative. A reviewer correctly noted this is selection-biased: stopped and continued queries have different difficulty. We therefore replace it with a counterfactual force-continue experiment (Appendix I) on the *same* stopped queries: forcing continuation lowers F1 (false-stop rate 7.8%, mean answer change  $-0.032$ ), which is the correct test. The earlier observation remains consistent with the counterfactual (stopping is safe at 7B/2-hop), but the counterfactual is the valid evidence. The signal-quality AUROC (gain predicting “force-continue helps”) is 0.503 on all queries, near chance—consistent with the low false-stop rate. At 1.5B/3-hop the proxy breaks (stopped 21.2% vs. continued 26.2%), consistent with the boundary analysis.

**J.0.5 Honest Boundaries.** We report settings where AquaRAG does *not* win. **(i) Trivial queries:** at 1.5B/1-hop, AquaRAG lowers cost by 22% but F1 drops  $0.341 \rightarrow 0.332$ . **(ii) Weak judge at depth:** at 1.5B/3-hop, No-Graph (F1 0.109) beats AquaRAG (0.091)—the 1.5B judge is too unreliable at depth 3. **(iii) Noisy judge on mixed streams:** on the 7B mixed stream, a tuned Fixed- $K=2$ ,  $B=8$  dominates AquaRAG (F1 0.474 vs 0.395), and even Oracle-depth is dominated (Section 4.5). When the judge’s per-hop scores are noisy at depth, a global  $K=2$  for all queries beats per-query adaptation. AquaRAG recovers on the stronger 14B judge (Section 4.4).

*The judge-quality scope condition.* These boundaries share a cause: (D3)’s signal  $t_k - t_{k-1}$  is a reliable proxy only when the judge’s scores are reliable. The 14B result and the counterfactual (App. E) bracket this: on a reliable judge, stopping is safe and beneficial; on a noisy judge at depth, it is not. This scope condition—not the controller—is the key determinant of when training-free GraphRAG adaptation succeeds. A practitioner should verify the judge discriminates informative from uninformative hops before deploying AquaRAG; otherwise a tuned global budget is preferable.

*Absolute performance and scope.* Our MetaQA F1 (0.18–0.53 at 2-hop by judge size) is far below near-saturated *trained* KGQA methods (EmbedKGQA, TransferNet). This is expected: those methods train task-specific graph reasoners, whereas we study *training-free* LLM-guided GraphRAG. Our claim is not that training-free GraphRAG matches supervised KGQA, but that *within* a fixed engine, the controller reduces cost at equal-or-better accuracy *when the judge is reliable*, and that this benefit transfers across engines (7B $\rightarrow$ 14B).

## K STRONG-ENGINE TRANSFER (14B)

To test whether AquaRAG’s benefit transfers to a stronger GraphRAG engine, we swap the 7B generator/judge for Qwen2.5-14B-Instruct (Qwen3-8B was unavailable in our environment) and re-run 2-hop retrieval ( $n=150$ , same KG and questions as the 7B main result). Table 19 reports the full comparison. AquaRAG improves F1 by 4.7% ( $0.506 \rightarrow 0.530$ ) while reducing cost by 15% ( $1487 \rightarrow 1266$  TOKS/Q)—a simultaneous F1 and cost win that does *not* hold under the 7B judge on the mixed stream (Section 4.5). This is the cleanest evidence for the judge-quality scope condition: the same controller, on the same KG, succeeds when the judge is reliable and fails when it is noisy at depth.